DATASETS

1) Registries and indexes

System **source-of-truth datasets**. Runtime reads them; humans maintain them. Every registry is **versioned, schema-validated, and referentially consistent**.

1.1 Standards registries

- **STANDARDS_INDEX**: all standards metadata (id, title, status, version, owner, scope tags, severity, effective date, supersedes/aliases).
- **APPLICABILITY_MAP**: deterministic rules selecting standards for a run (project type, domains, risk class, mode, target platform, executor type).
- **CONSTRAINT_PACK_INDEX**: group standards into packs; defines resolver order + precedence + conflict rules.
- **RESOLVER_ORDER_RULES** (if not embedded): explicit deterministic evaluation order for standards/packs (tie-breakers included).

1.2 Template registries

- **TEMPLATE_INDEX / REGISTRY**: all templates (id, version, category, required inputs, produces outputs, dependencies, compatible modes/profiles, render rules, completeness rules).
- **TEMPLATE_SELECTION_PROFILES**: named template sets for common builds (optional if you adopt build templates, but useful either way).
- **PLACEHOLDER_RULES_REGISTRY** (if not embedded): allowed placeholder/derive forms + validation rules.

1.3 Gate registries

- **GATE_INDEX**: gate definitions (gate_id, predicates, severity, remediation, required evidence types, pointer rules).
- **GATE_SET_PROFILES**: gate bundles by profile/mode (prototype vs hardened vs compliance, hotfix vs migration).

- **PREDICATE_OPERATOR_REGISTRY**: the allowed Gate DSL operators + argument schemas (prevents “invented predicates”).

1.4 Proof / evidence registries

- **PROOF_TYPE_REGISTRY**: allowed proof types (id, required fields, acceptable sources, retention, redaction rules).

- **EVIDENCE_SOURCE_REGISTRY**: allowed evidence sources (commands, CI artifacts, manual attestations, file hashes, repo refs).

- **EVIDENCE_POLICY_REGISTRY**: maps *gate_id → required proof types* + minimum metadata (this is what makes “must” enforceable).

1.5 Versioning & pinning registries

- **VERSION_REGISTRY**: current versions + compatibility (standards/templates/gates/schema versions), release notes, deprecations.

- **PINSET / LOCKFILE**: exact versions used for a run (standards snapshot version, template versions, gate set version, schema versions, toolchain versions).

- **SCHEMA_REGISTRY**: list of all artifact schemas (schema_id, version, path, applies_to artifact types) so schema drift is detectable.

1.6 Execution capability registries

- **ADAPTER_CAPABILITIES**: what the executor can do (run commands, network access, file IO, secrets access, build environment).

- **EXECUTOR_POLICY_PROFILE(S)**: constraints by executor type (internal agent vs external agent vs CI runner vs manual).

- **PACK/DOMAIN INDEX** (if used): pack IDs, produces/consumes, dependencies, ownership, stage mapping.

1.7 Run intent / operating mode registries

- **MAINTENANCE_MODE_REGISTRY (MM-xx)**: mode → active stages, active gates, proof strictness, required outputs.
- **RUN_PROFILE_REGISTRY** (if separate from MM): normal build intents (API/Web/Mobile/Fullstack/etc.) → default templates/standards/gates.

1.8 Core naming/term registries (prevents cross-doc drift)

- **TERMS_REGISTRY / GLOSSARY_INDEX**: canonical definitions + aliases + deprecation of terms.
- **ID_NAMESPACE_REGISTRY**: reserves prefixes + ID patterns (prevents collisions and silent renames).

---

2) Schemas and contracts

Validation surfaces that make outputs predictable and enforceable. Each artifact type has a schema and explicit required fields.

2.1 Artifact schemas (minimum set)

- **INTAKE**: submission, normalized input, submission record.
- **CANONICAL**: canonical spec, unknowns/assumptions, canonical diff.
- **STANDARDS**: applicability output, resolved standards snapshot.
- **TEMPLATES**: template selection result, render envelope, placeholder/derive syntax.
- **PLANNING**: work breakdown, acceptance map, build plan, plan unit.
- **ORDER/GATES**: dependency graph, gate report, evidence pointer.
- **VERIFICATION**: proof object, proof ledger, verification run result.
- **KIT**: kit manifest, entrypoint, bundle metadata.
- **STATE**: run snapshot, rollback snapshot, archive manifest.

- **GOV**: audit log entry, override record.

2.2 ID + naming contracts

- ID formats (SYS-xx, INT-xx, CAN-xx, STD-xx, TMP-xx, ORD-xx, PLAN-xx, VER-xx, KIT-xx, GATE-xx, PROOF-xx, RUN-xx, UNIT-xx).
- Canonical naming (registry/index/snapshot terms locked or aliased explicitly).
- Deterministic file paths + directory conventions per artifact type.

2.3 Stage I/O contracts

Per stage:

- consumes (artifact types + minimum fields)
- produces (artifact types + minimum fields)
- failure modes (hard stop vs warn)
- evidence requirements (what must be logged/proven)

2.4 Compatibility contracts

- version compatibility (template versions ↔ standards snapshot versions ↔ schema versions ↔ gate profiles)
- backward/forward compatibility rules for schema evolution (migration rules, deprecation windows)

---

3) Evidence and proof infrastructure

Turns claims into auditable facts and gates into objective checks.

3.1 Proof storage layout

- standardized paths for: run logs, command outputs, gate reports, proof objects, bundles/exports
- retention rules (keep forever vs prune vs redact)

3.2 Proof ledger

- append-only ledger (or append-first with explicit revision records)
- links proofs to: SPEC_INDEX items, work units, gates, runs, and bundle versions

3.3 Evidence pointer system

- uniform pointer formats: file path, log path, commit/PR ref, artifact hash ref
- resolver rules + validation (pointer must dereference; hash must match; paths must exist)

3.4 Proof capture standards

- what qualifies as proof for each gate
- minimum metadata: timestamp, tool version, environment, hashes, redaction flags

---

4) Test vectors and conformance suites

Verification backbone; “pass” must be meaningful.

4.1 Golden fixtures

- intake normalization goldens
- canonical spec goldens
- standards applicability/resolution goldens
- template rendering goldens

4.2 Predicate and gate tests

- unit tests for each predicate operator
- regression tests for known drift/failures

4.3 End-to-end golden runs

- full pipeline: INT → CAN → STD → TMP → PLAN → VER → KIT → STATE
- expected artifact set comparison (fixtures or generated compare)

4.4 Negative suites

- deliberately broken inputs that must fail at the correct stage/gate with correct remediation pointers

---

5) Tooling and CI plumbing

Makes AXION enforce itself continuously.

5.1 Standard command interface

- canonical commands: doctor, verify, run, stage, pack, repro
- stable exit codes + machine-readable reports

5.2 CI workflows

- PR checks: doctor + verify
- main checks: verify + reproducibility (if enabled)
- artifact upload: gate reports, logs, bundles

5.3 Local dev scripts

- one command: full cycle on a fixture
- one command: generate kit
- one command: repro run from manifest/pinset

5.4 Environment/toolchain control

- pinned tool versions or container runners
- deterministic builds (lockfiles, frozen deps, pinned compilers)

---

6) Documentation glue

Keeps the system navigable; prevents “tribal knowledge.”

6.1 Doc index + reading order

- authoritative doc ID list
- reading order per role (operator, template author, standards author, executor/agent)

6.2 Cross-doc dependency map

- doc → contract dependencies
- change impact notes (“edit X → must update Y/Z”)

6.3 Authoring rules

- where new standards/templates/gates live
- how to version them
- required sections for any new addition

6.4 Terminology authority

- canonical glossary/terms registry
- alias + rename policy that preserves references

---

7) Operational policies

Human governance that prevents unsafe overrides and unmanaged drift.

7.1 Override policy

- who can override
- which gates are never overridable
- override record requirements (reason, approver, expiry, scope)

7.2 Incident/hotfix procedures

- hotfix mode rules (what can be skipped vs never skipped)
- post-incident backfill requirements (proof, docs, regression tests)

7.3 Deprecation + breaking-change policy

- deprecation lifecycle (announce → warn → remove)
- breaking-change checklist (schema bumps, migration plans, compatibility windows)

7.4 Security + IP policy

- plagiarism/reuse allowlist + reuse log policy (internal + external agents)
- secrets handling policy (never in kits/logs)
- data handling for intake + archives (PII rules, retention, redaction)

8) Runtime module coverage map

Proves the docs are backed by real code and that the system is actually executable.

8.1 Doc → Runtime mapping

- **DOC_RUNTIME_MAP**: for each doc ID (SYS/INT/CAN/STD/TMP/ORD/PLAN/VER/KIT/STATE/GOV/EXEC), list:
    - owning runtime module(s)
    - CLI command(s) / API entrypoint(s)
    - produced artifacts
    - required tests/proofs

8.2 Capability status tracking

- **CAPABILITY_STATUS_LIST**: each capability marked:
    - implemented / partial / stubbed / deprecated
- **STUB_DETECTION_RULES**: definition of what counts as “stubbed” (NotImplemented, TODO-throw, missing handler)

8.3 Coverage requirements (what “covered” means)

- **Declared**: mapped to a runtime owner
- **Runnable**: can be executed via CLI/API and produces artifacts
- **Proven**: has tests or proof evidence and passes gates

8.4 Coverage gates

- gates that fail if:
    - required runtime pieces are missing

    - “stubbed” code exists in active paths
    - required proofs/tests for a capability are missing

---

9) Run profiles and build recipes

A deterministic “recipe layer” that selects the right standards/templates/gates/stages for a given build intent.

9.1 Build/run profile contract

- **RUN_PROFILE**: defines:
    - active stages
    - active gate set
    - default standards packs
    - template selection profile (or direct template set)
    - required outputs (artifacts + proofs)

9.2 Profile resolution rules

- **PROFILE_RESOLUTION_RULES**: deterministic selection when multiple profiles apply:
    - priority order
    - tie-breakers
    - overrides allowed (and how recorded)

9.3 Profile variants (common)

- prototype / standard / hardened / compliance
- internal agent vs external agent execution profiles

- mode-driven overrides (MM-xx modifies profile)

---

10) Control plane lifecycle specification

Makes the system governable as an actual runtime (not just scripts).

10.1 Run state machine

- **RUN_STATE_MODEL**: states and allowed transitions
    - queued → running → gated → failed → released → archived (example)
- **STAGE_TRANSITION_RULES**: what can advance, rerun, rollback, or resume

10.2 Persistence rules

- **STATE_PERSIST_POLICY**: what must be written at each transition:
    - run manifest
    - stage report
    - gate report
    - proof ledger updates
    - snapshots/pinsets

10.3 Operator actions and audit coupling

- **OP_ACTIONS_MODEL**: start/stop/resume/override/approve
- **AUDIT_BINDING_RULES**: every operator action writes an audit record with required fields

10.4 Failure semantics

- **HARD_STOP_RULES**: what failures block progression
- **RETRY/RERUN_POLICY**: safe reruns, idempotency expectations

---

11) Execution security boundary model

The mechanical constraints that make “internal vs external agent” safe and enforceable.

11.1 Capability sandboxing

- **EXECUTION_SANDBOX_RULES**: per executor type:
    - allowed filesystem roots
    - allowed network access
    - allowed commands/tooling
    - secrets access rules

11.2 Data boundary rules

- **DATA_CLASSIFICATION_RULES**: PII/secret/internal/public classifications
- **REDACTION_POLICY**: what must be removed from:
    - logs
    - kits
    - archives
    - proof objects

11.3 External agent restrictions

- **EXTERNAL_AGENT_CONSTRAINTS**:

    - no access to knowledge library unless explicitly allowed
    - reuse allowlist required
    - minimal artifact exposure in kits (principle of least privilege)

11.4 Security gates

- gates that fail if:
    - disallowed paths are accessed
    - secrets/PII appear in logs or kits
    - executor capability claims don't match actual behavior

12) Release and distribution discipline

Enterprise-grade supply-chain guarantees for shipped kits and runtime releases. Add only if you want tamper resistance, provenance, and formal release governance.

12.1 Release artifact set

Defines what constitutes a "release" and what must be produced.

- **RELEASE_MANIFEST**: release_id, included bundle_ids, pinset, schema versions, gate set used, compatibility notes
- **RELEASE_NOTES**: changes, breaking changes, migrations, deprecations
- **RELEASE_ATTESTATIONS** (optional but common): provenance metadata for how the bundle was produced

12.2 Signing and integrity

Ensures distributed bundles/manifests can't be tampered with silently.

- **SIGNATURE_POLICY**: what must be signed (bundle, manifest, pinset, gate report)
- **KEY_MANAGEMENT_RULES**: key storage, rotation, revocation, who can sign

- **INTEGRITY_CHECK_RULES**: verify signatures/hashes on import and repro

12.3 Provenance and SBOM (optional but strong)

Makes builds auditable at the dependency level.

- **PROVENANCE_RECORD**: toolchain versions, environment fingerprint, build commands, source commit refs
- **SBOM**: dependency inventory (runtime + build-time)
- **DEPENDENCY_RISK_POLICY**: denylist/allowlist, license compatibility, CVE thresholds

12.4 Distribution channels and access control

Controls who gets which bundles and what they contain.

- **DISTRIBUTION_RULES**: internal vs external bundle variants, redaction rules per channel
- **ACCESS_CONTROL_POLICY**: who can download/export bundles, logging of access
- **RETENTION_POLICY (release-level)**: retention windows for released artifacts

12.5 Release gates

Release-only gates that must pass before anything is labeled “released.”

- **GATE-REL-01**: all required stage artifacts present + pinned
- **GATE-REL-02**: reproducibility check passes (pack + repro match)
- **GATE-REL-03**: signatures valid (if signing enabled)
- **GATE-REL-04**: SBOM/provenance present (if required)
- **GATE-REL-05**: IP/security checks pass (reuse log + secrets scan)

Registries and indexes

Registries and indexes

System **source-of-truth datasets**. Runtime reads them; humans maintain them. Every registry is **versioned, schema-validated, and referentially consistent**.

1.1 Standards registries

- **STANDARDS_INDEX**: all standards metadata (id, title, status, version, owner, scope tags, severity, effective date, supersedes/aliases).
- **APPLICABILITY_MAP**: deterministic rules selecting standards for a run (project type, domains, risk class, mode, target platform, executor type).
- **CONSTRAINT_PACK_INDEX**: group standards into packs; defines resolver order + precedence + conflict rules.
- **RESOLVER_ORDER_RULES** (if not embedded): explicit deterministic evaluation order for standards/packs (tie-breakers included).

1.2 Template registries

- **TEMPLATE_INDEX / REGISTRY**: all templates (id, version, category, required inputs, produces outputs, dependencies, compatible modes/profiles, render rules, completeness rules).
- **TEMPLATE_SELECTION_PROFILES**: named template sets for common builds (optional if you adopt build templates, but useful either way).
- **PLACEHOLDER_RULES_REGISTRY** (if not embedded): allowed placeholder/derive forms + validation rules.

1.3 Gate registries

- **GATE_INDEX**: gate definitions (gate_id, predicates, severity, remediation, required evidence types, pointer rules).
- **GATE_SET_PROFILES**: gate bundles by profile/mode (prototype vs hardened vs compliance, hotfix vs migration).

- **PREDICATE_OPERATOR_REGISTRY**: the allowed Gate DSL operators + argument schemas (prevents “invented predicates”).

1.4 Proof / evidence registries

- **PROOF_TYPE_REGISTRY**: allowed proof types (id, required fields, acceptable sources, retention, redaction rules).

- **EVIDENCE_SOURCE_REGISTRY**: allowed evidence sources (commands, CI artifacts, manual attestations, file hashes, repo refs).

- **EVIDENCE_POLICY_REGISTRY**: maps *gate_id → required proof types* + minimum metadata (this is what makes “must” enforceable).

1.5 Versioning & pinning registries

- **VERSION_REGISTRY**: current versions + compatibility (standards/templates/gates/schema versions), release notes, deprecations.

- **PINSET / LOCKFILE**: exact versions used for a run (standards snapshot version, template versions, gate set version, schema versions, toolchain versions).

- **SCHEMA_REGISTRY**: list of all artifact schemas (schema_id, version, path, applies_to artifact types) so schema drift is detectable.

1.6 Execution capability registries

- **ADAPTER_CAPABILITIES**: what the executor can do (run commands, network access, file IO, secrets access, build environment).

- **EXECUTOR_POLICY_PROFILE(S)**: constraints by executor type (internal agent vs external agent vs CI runner vs manual).

- **PACK/DOMAIN INDEX** (if used): pack IDs, produces/consumes, dependencies, ownership, stage mapping.

1.7 Run intent / operating mode registries

- **MAINTENANCE_MODE_REGISTRY (MM-xx)**: mode → active stages, active gates, proof strictness, required outputs.

- **RUN_PROFILE_REGISTRY** (if separate from MM): normal build intents (API/Web/Mobile/Fullstack/etc.) → default templates/standards/gates.

1.8 Core naming/term registries (prevents cross-doc drift)

- **TERMS_REGISTRY / GLOSSARY_INDEX**: canonical definitions + aliases + deprecation of terms.

- **ID_NAMESPACE_REGISTRY**: reserves prefixes + ID patterns (prevents collisions and silent renames).

Standards registries

Standards registries

- **STANDARDS_INDEX**: all standards metadata (id, title, status, version, owner, scope tags, severity, effective date, supersedes/aliases).

- **APPLICABILITY_MAP**: deterministic rules selecting standards for a run (project type, domains, risk class, mode, target platform, executor type).

- **CONSTRAINT_PACK_INDEX**: group standards into packs; defines resolver order + precedence + conflict rules.

- **RESOLVER_ORDER_RULES** (if not embedded): explicit deterministic evaluation order for standards/packs (tie-breakers included).

STANDARDS_INDEX

APPLICABILITY_MAP

CONSTRAINT_PACK_INDEX

RESOLVER_ORDER_RULES

Template registries

Template registries

- **TEMPLATE_INDEX / REGISTRY**: all templates (id, version, category, required inputs, produces outputs, dependencies, compatible modes/profiles, render rules, completeness rules).

- **TEMPLATE_SELECTION_PROFILES**: named template sets for common builds (optional if you adopt build templates, but useful either way).

- **PLACEHOLDER_RULES_REGISTRY** (if not embedded): allowed placeholder/derive forms + validation rules.

Tab 41

Tab 42

Tab 43

Gate registries

Gate registries

- **GATE_INDEX**: gate definitions (gate_id, predicates, severity, remediation, required evidence types, pointer rules).

- **GATE_SET_PROFILES**: gate bundles by profile/mode (prototype vs hardened vs compliance, hotfix vs migration).

- **PREDICATE_OPERATOR_REGISTRY**: the allowed Gate DSL operators + argument schemas (prevents “invented predicates”).

Tab 44

Tab 45

Tab 46

Proof / evidence registries

Proof / evidence registries

- **PROOF_TYPE_REGISTRY**: allowed proof types (id, required fields, acceptable sources, retention, redaction rules).

- **EVIDENCE_SOURCE_REGISTRY**: allowed evidence sources (commands, CI artifacts, manual attestations, file hashes, repo refs).

- **EVIDENCE_POLICY_REGISTRY**: maps *gate_id → required proof types* + minimum metadata (this is what makes “must” enforceable).

Tab 47

Tab 48

Tab 49

Versioning & pinning registries

Versioning & pinning registries

- **VERSION_REGISTRY**: current versions + compatibility (standards/templates/gates/schema versions), release notes, deprecations.

- **PINSET / LOCKFILE**: exact versions used for a run (standards snapshot version, template versions, gate set version, schema versions, toolchain versions).

- **SCHEMA_REGISTRY**: list of all artifact schemas (schema_id, version, path, applies_to artifact types) so schema drift is detectable.

Tab 50

Tab 51

Tab 52

Execution capability registries

Execution capability registries

- **ADAPTER_CAPABILITIES**: what the executor can do (run commands, network access, file IO, secrets access, build environment).

- **EXECUTOR_POLICY_PROFILE(S)**: constraints by executor type (internal agent vs external agent vs CI runner vs manual).

- **PACK/DOMAIN INDEX** (if used): pack IDs, produces/consumes, dependencies, ownership, stage mapping.

Tab 53

Tab 54

Tab 55

Run intent / operating mode registries

Run intent / operating mode registries

- **MAINTENANCE_MODE_REGISTRY (MM-xx)**: mode → active stages, active gates, proof strictness, required outputs.

- **RUN_PROFILE_REGISTRY** (if separate from MM): normal build intents (API/Web/Mobile/Fullstack/etc.) → default templates/standards/gates.

Tab 56

Tab 57

Core naming/term registries

Core naming/term registries (prevents cross-doc drift)

- **TERMS_REGISTRY / GLOSSARY_INDEX**: canonical definitions + aliases + deprecation of terms.

- **ID_NAMESPACE_REGISTRY**: reserves prefixes + ID patterns (prevents collisions and silent renames).

Tab 58

Tab 59

Schemas and contracts

Schemas and contracts

Validation surfaces that make outputs predictable and enforceable. Each artifact type has a schema and explicit required fields.

2.1 Artifact schemas (minimum set)

- **INTAKE**: submission, normalized input, submission record.
- **CANONICAL**: canonical spec, unknowns/assumptions, canonical diff.
- **STANDARDS**: applicability output, resolved standards snapshot.
- **TEMPLATES**: template selection result, render envelope, placeholder/derive syntax.
- **PLANNING**: work breakdown, acceptance map, build plan, plan unit.
- **ORDER/GATES**: dependency graph, gate report, evidence pointer.
- **VERIFICATION**: proof object, proof ledger, verification run result.
- **KIT**: kit manifest, entrypoint, bundle metadata.
- **STATE**: run snapshot, rollback snapshot, archive manifest.
- **GOV**: audit log entry, override record.

2.2 ID + naming contracts

- ID formats (SYS-xx, INT-xx, CAN-xx, STD-xx, TMP-xx, ORD-xx, PLAN-xx, VER-xx, KIT-xx, GATE-xx, PROOF-xx, RUN-xx, UNIT-xx).
- Canonical naming (registry/index/snapshot terms locked or aliased explicitly).
- Deterministic file paths + directory conventions per artifact type.

2.3 Stage I/O contracts

Per stage:

- consumes (artifact types + minimum fields)
- produces (artifact types + minimum fields)
- failure modes (hard stop vs warn)
- evidence requirements (what must be logged/proven)

2.4 Compatibility contracts

- version compatibility (template versions ↔ standards snapshot versions ↔ schema versions ↔ gate profiles)
- backward/forward compatibility rules for schema evolution (migration rules, deprecation windows)

Artifact schemas

2.1 Artifact schemas (minimum set)

- **INTAKE**: submission, normalized input, submission record.
- **CANONICAL**: canonical spec, unknowns/assumptions, canonical diff.
- **STANDARDS**: applicability output, resolved standards snapshot.
- **TEMPLATES**: template selection result, render envelope, placeholder/derive syntax.
- **PLANNING**: work breakdown, acceptance map, build plan, plan unit.
- **ORDER/GATES**: dependency graph, gate report, evidence pointer.
- **VERIFICATION**: proof object, proof ledger, verification run result.
- **KIT**: kit manifest, entrypoint, bundle metadata.
- **STATE**: run snapshot, rollback snapshot, archive manifest.
- **GOV**: audit log entry, override record.

Tab 60

Tab 61

Tab 62

Tab 63

Tab 64

Tab 65

Tab 66

Tab 67

Tab 68

Tab 69

ID + naming contracts

ID + naming contracts

- ID formats (SYS-xx, INT-xx, CAN-xx, STD-xx, TMP-xx, ORD-xx, PLAN-xx, VER-xx, KIT-xx, GATE-xx, PROOF-xx, RUN-xx, UNIT-xx).
- Canonical naming (registry/index/snapshot terms locked or aliased explicitly).
- Deterministic file paths + directory conventions per artifact type.

Tab 70

Tab 71

Tab 72

Stage I/O contracts

Stage I/O contracts

Per stage:

- consumes (artifact types + minimum fields)
- produces (artifact types + minimum fields)
- failure modes (hard stop vs warn)
- evidence requirements (what must be logged/proven)

Tab 73

Tab 74

Tab 75

Tab 76

Compatibility contracts

Compatibility contracts

- version compatibility (template versions ↔ standards snapshot versions ↔ schema versions ↔ gate profiles)

- backward/forward compatibility rules for schema evolution (migration rules, deprecation windows)

Tab 77

Tab 78

Evidence and proof infrastructure

Evidence and proof infrastructure

Purpose: make every “PASS” defensible by turning outputs into **auditable evidence** and by making gates evaluate **structured proof**, not vibes.

3.1 Proof storage layout

You need a **deterministic storage model** so pointers are stable and reproducible.

Required directories (conceptual)

- **Runs**
  - `runs/<run_id>/`
    Contains *everything produced during a run*.
- **Proof objects**
  - `proof/<proof_id>.json` (or `runs/<run_id>/proof/…`)
- **Logs**
  - `runs/<run_id>/logs/<command_id>.log`
- **Gate reports**
  - `runs/<run_id>/gate_report.json` (+ per-gate optional)
- **Selection outputs**
  - `runs/<run_id>/selection/standards.json`
  - `runs/<run_id>/selection/templates.json`
- **Bundles**
  - `bundles/<bundle_id>/` (or `runs/<run_id>/bundle/…`)

Required invariants

- every run has a unique `run_id`
- run output paths are deterministic and never ambiguous
- artifacts referenced by pointers must be in known locations

Retention rules (minimum)

Define retention classes:

- **KEEP**: always retain (gate reports, manifests, pinsets, proof ledger entries)
- **PRUNE_OK**: can prune after N days (raw logs, intermediate debug output)
- **REDACT_REQUIRED**: must redact secrets/PII before persistence (intake artifacts, raw prompts if stored)

3.2 Proof ledger

The proof ledger is the **audit spine**: it tells you *what proof exists* and *what it proves*.

Minimum ledger behavior

- **Append-only** preferred
  If updates are needed: use **revision records**, not in-place mutation.
- Ledger entries are immutable once published for a released run/bundle.

Minimum linkage fields (must exist)

Each proof entry must link to at least:

- **run_id**
- **proof_id**

- **proof_type**
- **applies_to**
    - one or more of: `spec_item_id`, `work_unit_id`, `gate_id`, `bundle_id`
- **source**
    - command / file / commit / attestation

Required invariants

- every `proof_id` is unique
- every referenced `spec_item_id / work_unit_id / gate_id` exists
- proof types must exist in `PROOF_TYPE_REGISTRY`

3.3 Evidence pointer system

Pointers are how gates and reports refer to evidence **without embedding it**.

Allowed pointer formats (minimum set)

- **file:** `file:axion/runs/RUN-.../logs/test.log`
- **log:** `log:RUN-...:test` (resolves to file path)
- **commit:** `commit:<sha>`
- **pr:** `pr:<number>`
- **hash:** `sha256:<hex>:file:<path>` (hash + location)
- **artifact:** `artifact:<artifact_id>` (resolves via manifest)

Resolver rules (must exist)

- dereference must succeed (path exists / commit exists)
- hash pointers must match content (or fail the gate)
- pointers must be restricted to allowed roots (prevents leaking secrets paths)

Required invariants

- pointers are stable for the life of the run/bundle
- pointer schemes are finite and schema-validated (no freeform strings)

3.4 Proof capture standards

This defines what counts as acceptable proof, and ensures proofs are comparable.

Proof qualification rules (minimum)

For each **gate_id**, define:

- required proof types
- minimum evidence sources (command output vs manual attest vs artifact hash)
- minimum coverage (e.g., “all required commands pass”)
- staleness rules (proof must be from current run or pinned compatible run)

Minimum proof metadata (must be captured)

- timestamp
- tool/version (node version, test runner version, build tool version)
- environment (ci/local, OS, container hash if applicable)

- hashes (where relevant)
- redaction flags (whether content was redacted)

Required invariants

- proof must be attributable to a run (run_id)
- proof must be attributable to a source (command/file/commit)
- proof must satisfy the registry schema for its proof_type

What the system “needs” vs “nice-to-have”

Needs (to be real)

- deterministic run folder layout
- proof ledger with run_id + linkages
- validated pointer formats + resolver
- gate→proof requirements defined somewhere (evidence policy)

Nice-to-have (later)

- signed proofs / attestations
- SBOM + provenance (SLSA-style)
- tamper-evident audit chain (hash chaining the ledger)

Proof storage layout

Proof storage layout

You need a **deterministic storage model** so pointers are stable and reproducible.

Required directories (conceptual)

- **Runs**
  - `runs/<run_id>/`
    Contains *everything produced during a run*.
- **Proof objects**
  - `proof/<proof_id>.json` (or `runs/<run_id>/proof/…`)
- **Logs**
  - `runs/<run_id>/logs/<command_id>.log`
- **Gate reports**
  - `runs/<run_id>/gate_report.json` (+ per-gate optional)
- **Selection outputs**
  - `runs/<run_id>/selection/standards.json`
  - `runs/<run_id>/selection/templates.json`
- **Bundles**
  - `bundles/<bundle_id>/` (or `runs/<run_id>/bundle/…`)

Required invariants

- every run has a unique `run_id`
- run output paths are deterministic and never ambiguous

- artifacts referenced by pointers must be in known locations

Retention rules (minimum)

Define retention classes:

- **KEEP**: always retain (gate reports, manifests, pinsets, proof ledger entries)
- **PRUNE_OK**: can prune after N days (raw logs, intermediate debug output)
- **REDACT_REQUIRED**: must redact secrets/PII before persistence (intake artifacts, raw prompts if stored)

Proof ledger

Proof ledger

The proof ledger is the **audit spine**: it tells you *what proof exists* and *what it proves*.

Minimum ledger behavior

- **Append-only** preferred
  If updates are needed: use **revision records**, not in-place mutation.

- Ledger entries are immutable once published for a released run/bundle.

Minimum linkage fields (must exist)

Each proof entry must link to at least:

- **run_id**

- **proof_id**

- **proof_type**

- **applies_to**

  - one or more of: `spec_item_id`, `work_unit_id`, `gate_id`, `bundle_id`

- **source**

  - command / file / commit / attestation

Required invariants

- every `proof_id` is unique

- every referenced `spec_item_id / work_unit_id / gate_id` exists

- proof types must exist in `PROOF_TYPE_REGISTRY`

Evidence pointer system

Evidence pointer system

Pointers are how gates and reports refer to evidence **without embedding it**.

Allowed pointer formats (minimum set)

- **file:** `file:axion/runs/RUN-.../logs/test.log`
- **log:** `log:RUN-...:test` (resolves to file path)
- **commit:** `commit:<sha>`
- **pr:** `pr:<number>`
- **hash:** `sha256:<hex>:file:<path>` (hash + location)
- **artifact:** `artifact:<artifact_id>` (resolves via manifest)

Resolver rules (must exist)

- dereference must succeed (path exists / commit exists)
- hash pointers must match content (or fail the gate)
- pointers must be restricted to allowed roots (prevents leaking secrets paths)

Required invariants

- pointers are stable for the life of the run/bundle
- pointer schemes are finite and schema-validated (no freeform strings)

Proof capture standards

Proof capture standards

This defines what counts as acceptable proof, and ensures proofs are comparable.

Proof qualification rules (minimum)

For each **gate_id**, define:

- required proof types
- minimum evidence sources (command output vs manual attest vs artifact hash)
- minimum coverage (e.g., “all required commands pass”)
- staleness rules (proof must be from current run or pinned compatible run)

Minimum proof metadata (must be captured)

- timestamp
- tool/version (node version, test runner version, build tool version)
- environment (ci/local, OS, container hash if applicable)
- hashes (where relevant)
- redaction flags (whether content was redacted)

Required invariants

- proof must be attributable to a run (run_id)
- proof must be attributable to a source (command/file/commit)
- proof must satisfy the registry schema for its proof_type

What the system “needs” vs “nice-to-have”

Needs (to be real)

- deterministic run folder layout
- proof ledger with run_id + linkages
- validated pointer formats + resolver
- gate→proof requirements defined somewhere (evidence policy)

Nice-to-have (later)

- signed proofs / attestations
- SBOM + provenance (SLSA-style)
- tamper-evident audit chain (hash chaining the ledger)

Test vectors and conformance suites

Test vectors and conformance suites

Purpose: validate **determinism, correctness, enforcement, and drift resistance** across the pipeline.

AXION needs **four test layers**:

1. **Golden fixtures** (known correct inputs/outputs)
2. **Operator/predicate tests** (Gate DSL correctness)
3. **End-to-end golden runs** (full pipeline behavior)
4. **Negative suites** (failures happen in the right place, with the right reasons)

4.1 Golden fixtures

Golden fixtures are **locked examples** with expected outputs. They prove each stage produces deterministic artifacts.

A) Intake normalization goldens

What they test

- submission → normalized input is deterministic and complete
- normalization handles edge formats (missing fields, weird shapes)

Minimum fixtures

- `INT-GOLD-01` minimal valid submission
- `INT-GOLD-02` messy submission (extra fields, inconsistent casing)
- `INT-GOLD-03` risky submission (PII/secrets present → redaction rules)

- `INT-GOLD-04` invalid submission (must fail with specific code)

Expected outputs

- normalized input JSON
- submission record JSON
- validation error report for invalid cases

B) Canonical spec goldens

What they test

- normalized input → canonical spec is stable
- unknowns/assumptions are captured deterministically
- canonical diffs are stable

Minimum fixtures

- `CAN-GOLD-01` simple app canonical
- `CAN-GOLD-02` app with unknowns (missing requirement data)
- `CAN-GOLD-03` update case (prior canonical pinned → diff produced)

Expected outputs

- canonical spec JSON
- unknowns JSON (if applicable)
- canonical diff output

C) Standards applicability/resolution goldens

What they test

- applicability map selects the same standards for the same run context
- resolution order is deterministic
- conflicts resolve consistently
- snapshot format is correct and pinned

Minimum fixtures

- `STD-GOLD-01` baseline profile (prototype)
- `STD-GOLD-02` hardened profile (production/compliance)
- `STD-GOLD-03` conflicting standards case (tests precedence rules)

Expected outputs

- applicability output (selected standards + rationale)
- resolved standards snapshot (final resolved set + versions + ordering)

D) Template rendering goldens

What they test

- selection chooses correct templates deterministically
- render output matches envelope spec
- placeholder/derive rules behave correctly
- completeness checks work

Minimum fixtures

- `TMP-GOLD-01` one template render
- `TMP-GOLD-02` multiple templates + dependencies
- `TMP-GOLD-03` missing required fields triggers completeness failure
- `TMP-GOLD-04` placeholder syntax edge cases

Expected outputs

- template selection result
- render envelope(s)
- completeness report

4.2 Predicate and gate tests

This layer makes gates trustworthy. If your gate engine is wrong, nothing matters.

A) Predicate operator unit tests

What they test

- every operator's behavior is correct on PASS and FAIL
- error messaging is stable and includes pointers

Minimum operator suite

- file existence, non-empty checks
- schema validation checks
- coverage operators
- ledger requirements (`require_fields`, `targets_implemented`)

- command pass/fail operators
- pointer resolution operators

Required outputs

- structured predicate result objects
- stable remediation strings (or stable codes)

B) Regression tests for known drift/failures

What they test

- previously fixed failure modes stay fixed

Examples

- “template selection changed when order of inputs changed”
- “standards conflict resolved differently run-to-run”
- “gate report missing pointers”
- “proof ledger mutated instead of revision appended”

4.3 End-to-end golden runs

This is the “does AXION actually work as a system?” test.

What they test (minimum)

- stage order INT → CAN → STD → TMP → PLAN → VER → KIT → STATE
- artifacts produced at each stage match schema
- run outputs are deterministic for the same inputs + pinset

- required gates pass and produce expected reports

Golden run types (minimum set)

- `E2E-GOLD-01` happy path (minimal project)
- `E2E-GOLD-02` strict profile (hardened gates on)
- `E2E-GOLD-03` update run (prior kit pinned → diff + reproducibility)

Expected artifact set comparison

You should compare either:

- **fixtures checked in** (expected outputs stored), or
- **generated compare** (re-run twice and compare hashes)

Minimum checks

- kit manifest exists and matches schema
- standards snapshot pinned and referenced
- template render envelopes exist
- gate report exists and is PASS
- state snapshot exists

4.4 Negative suites

These prove failures are **correctly located** and **actionable**.

What they test (minimum)

For each failure, you validate:

- which stage fails
- which gate fails
- the failure reason is correct
- remediation is specific (not generic)
- pointers resolve to real evidence/artifacts

Minimum negative suite cases

- `NEG-01` intake missing required fields → fails INT validation (not later)
- `NEG-02` standards conflict without precedence rule → fails STD resolution
- `NEG-03` template missing required placeholder values → fails TMP completeness
- `NEG-04` verification command fails → fails VER gate with log pointer
- `NEG-05` kit missing required artifact → fails KIT validation
-

Tab 22

Tab 23

Tab 24

Tab 25

Tooling and CI plumbing

Tooling and CI plumbing

Purpose: make enforcement **automatic, repeatable, and drift-resistant** across local dev and CI.

5.1 Standard command interface

AXION needs a **small, stable CLI surface** that every automation can call.

Required commands (minimum)

- `doctor` — checks repo health + required files + JSON/schema validity + registry integrity
- `verify` — runs verification commands + gate evaluation + produces gate report
- `run` — executes a full pipeline run (or “next action” loop)
- `stage <id>` — run a single stage (for debugging or targeted reruns)
- `pack` — produces a kit/bundle (manifest + entrypoint + exported archive)
- `repro` — reproduces a run from a manifest/pinset (determinism proof)

Required CLI guarantees

- **Stable exit codes**
  - `0` = PASS
  - `2` = FAIL (gates failed / contract invalid)
  - `1` = ERROR (unexpected exception / runtime failure)
- **Machine-readable outputs**

    - always writes `gate_report.json`
    - writes a `run_log.jsonl` (or similar) for trace
    - prints a short summary to stdout (but stdout isn't the source of truth)
- **Deterministic output locations**
    - every command knows where it writes artifacts (so CI can upload them)

5.2 CI workflows

CI is where AXION becomes non-optional.

PR checks (minimum)

- Run **`doctor`** first (fast fail on broken repo/system)
- Run **`verify`** second (enforces gates + proofs)
- Block merge if either fails

Main branch checks (minimum)

- Run **`verify`** again (ensures main is always valid)
- Optional but high-value: **reproducibility check**
    - run `pack` then `repro` on the same inputs/pinset
    - compare hashes/manifests to prove determinism

Artifact upload (minimum)

On failure, CI must upload:

- `gate_report.json`
- relevant logs (command outputs)
- (optional) bundle/manifest if pack ran
  This is critical because it makes failures debuggable without rerunning locally.

Required invariants

- CI must use a **clean environment** (no cached junk influencing results)
- CI must use the same tool versions as local (or pinned containers)

5.3 Local dev scripts

These are “human throughput” tools: they make AXION usable day-to-day.

Required scripts (minimum)

- **`axion:cycle`** — runs a full cycle on a known fixture (INT→…→KIT→STATE)
- **`axion:kit`** — generates a kit/bundle from a fixture or current project
- **`axion:repro`** — reproduces the last run using its pinset/manifest
- **`axion:clean`** — clears transient run outputs safely (does not delete pinned artifacts)

Required invariants

- “one command, no questions” for the common loop
- local runs produce the same artifacts/paths as CI (so debugging is consistent)

5.4 Environment and toolchain control

This prevents the #1 CI/local mismatch issue: toolchain drift.

Minimum controls (pick one strategy)

Strategy A — Pin tool versions directly

- Node version pinned
- package lockfiles enforced (`npm ci` equivalent)
- compiler versions pinned (tsc, etc.)

Strategy B — Container runners

- a container image with:
  - node/toolchain versions baked in
  - OS-level deps fixed
- CI and local both run in same container profile

Deterministic build requirements

- lockfiles required and enforced
- “frozen deps” behavior (no auto-updates)
- stable ordering and normalization of outputs (no nondeterministic JSON ordering, timestamps isolated)
-

Tab 27

Tab 28

Tab 29

Tab 30

Documentation glue

Documentation glue

Purpose: make AXION **navigable, teachable, and change-safe**. This is how you prevent “the system works only because you remember everything.”

6.1 Doc index + reading order

You need a **single authoritative doc index** that answers: *what exists, where it lives, what it is for, and what depends on it.*

Doc Index (minimum fields)

For each doc:

- `doc_id` (SYS-01, STD-03, TMP-02…)
- `title`
- `status` (draft/locked/deprecated)
- `version`
- `owner`
- `source_path` (where the doc lives)
- `contract_surface` (what it defines: schema, policy, format, etc.)
- `depends_on` (doc_ids)
- `produces` / `governs` (which artifacts or registries it defines)

Reading order per role (minimum sets)

These are “paths,” not a giant list.

- **Operator path**
  - SYS → ORD → VER → KIT → STATE → GOV
- **Template author path**
  - TMP → CAN → STD → ORD (completeness rules) → KIT (render envelope rules)
- **Standards author path**
  - STD → APPLICABILITY → ORD (gate mapping) → VER (proof rules) → GOV (policy boundaries)
- **Executor/agent path**
  - CAN → STD snapshot → TMP outputs → PLAN units → VER commands → KIT entrypoint

Required invariants

- doc IDs are unique and never reused
- locked docs have a change protocol (version bump, migration note, etc.)
- the reading order lists always reference valid doc IDs

6.2 Cross-doc dependency map

You need an explicit graph of “if you change X, what breaks.”

Two dependency maps you actually need

1. **Doc → Doc dependencies**

- each doc lists upstream docs it relies on
- used for change impact

2. **Doc → Runtime/Contract dependencies**

- maps a doc to the runtime components, schemas, registries, and gates it governs
- used to ensure code coverage and compliance

Change impact notes (what they are)

For each doc (especially “locked” ones), maintain a “change impact” block:

- **If you edit this doc, you must review/update:**
    - affected registries
    - affected schemas
    - affected gates
    - affected templates
    - affected runtime modules
- **Expected migration steps**
- **Compatibility constraints** (what must stay backward compatible)

Required invariants

- no hidden dependencies (if runtime uses a contract, it must be in the map)
- dependency map must not drift from reality (it’s updated during compliance passes)

6.3 Authoring rules

These rules keep additions consistent and prevent the “random doc sprawl.”

Placement rules (minimum)

- where new **standards** live
- where new **templates** live
- where new **gates** live
- where new **schemas** live
- where new **modes/profiles** live

Versioning rules (minimum)

- when to bump version (format change vs content tweak)
- how to deprecate (announce → alias → remove)
- what “locked” means and how to change locked docs

Required sections for any new addition

This is crucial. Every new standard/template/gate should include:

- ID + purpose
- scope/applicability
- inputs/outputs (if relevant)
- required proofs/evidence (if enforceable)
- dependencies (what it assumes exists)
- examples (optional but helpful)
- migration notes if it replaces something

Required invariants

- no additions without registry entries

- no additions without schema references where applicable
- no changes to locked contracts without a compatibility note

---

6.4 Terminology authority

This stops the biggest source of drift: definitions changing between docs.

Canonical glossary / terms registry (minimum fields)

For each term:

- `term`
- `definition` (canonical)
- `aliases` (allowed synonyms)
- `deprecated_aliases` (old terms, with replacement)
- `first_defined_in` (doc_id)
- `used_in` (list of doc_ids, optional)

Alias + rename policy (must exist)

Rules:

- a term can have only **one canonical definition**
- renames must preserve backward references via alias mapping
- old terms remain valid for a deprecation window (so docs don't break)
- templates and registries should reference canonical terms, not aliases

Required invariants

- no doc is allowed to redefine glossary terms
- every doc uses canonical terms or declared aliases
- term drift triggers a compliance fail (at least a warning gate)
-

Tab 33

Tab 34

Tab 35

Tab 36

Operational policies

Operational policies

Purpose: define **who can do what**, **when rules can be relaxed**, and **how the system recovers** while preserving auditability and preventing drift.

7.1 Override policy

Overrides are controlled exceptions. The policy exists so “we had to ship” doesn’t become permanent drift.

A) Override authority (who can override)

Define roles (minimum):

- **Operator**: can run modes, view reports, request overrides
- **Approver**: can approve overrides within scope
- **Owner**: responsible for a domain/standard/template (can approve within their surface)
- **Admin**: can approve emergency overrides (still recorded)

B) Override scope (what can be overridden)

Overrides must specify:

- `gate_id` (exact gate)
- `run_id` (exact run)
- `scope` (what the override applies to: unit_id, pack, template_id, artifact path)
- `expiry` (time or run count)
- `risk_class` (why override is acceptable)

C) Never-overridable gates

You should explicitly mark some gates as **NON_OVERRIDABLE**. Typical examples:

- schema validity for core artifacts (manifest, standards snapshot, gate report)
- evidence pointer integrity (no broken pointers)
- secrets/PII leakage gates
- audit log write failure
- pinset/lockfile missing on a “release” run

(You choose the list—policy requires that a list exists.)

D) Override record requirements (must be written)

Every override produces an **override record** that includes:

- requester + approver identity
- timestamp
- reason (structured + free text)
- affected IDs (run/unit/template/gate)
- expiry
- remediation plan (what will be done to remove the override)

E) Override consequences

Overrides must trigger:

- an automatic **follow-up task** (backfill proof / fix root cause)
- elevated logging in audit ledger
- mode escalation if needed (e.g., forcing a post-hotfix compliance run)

7.2 Incident / hotfix procedures

Hotfix mode exists to ship safely under pressure without tearing down standards.

A) Hotfix mode rules

Hotfix policy must define:

- what stages are **skippable**
- what gates are **still mandatory**
- what proofs are **still mandatory**
- what outputs must be created

Typical “never skip” items

- intake validity (don’t ship unvalidated inputs)
- gate report generation (must still be accountable)
- security/IP checks (no secrets, no plagiarism)
- proof ledger update (must log what happened)

B) Incident record

Every incident/hotfix run creates:

- incident summary (what/why/impact)
- scope of change (files/units affected)
- override list (if any)
- rollback plan reference

C) Post-incident backfill (mandatory)

After hotfix, you require a backfill run that:

- re-runs full verify gates
- adds missing proofs
- updates docs/templates/standards if the hotfix revealed a gap
- adds regression tests for the failure cause

Hotfix is “temporary permission,” not a permanent state.

7.3 Deprecation + breaking-change policy

This prevents “silent breakage” across templates, schemas, and registries.

A) Deprecation lifecycle

Minimum lifecycle:

1. **Announce** (registry marks deprecated; replacement is named)
2. **Warn** (gates emit warnings; still allowed)
3. **Block new usage** (can’t be selected for new runs; old runs still repro)
4. **Remove** (only after migration window)

B) Breaking-change checklist (must exist)

For any breaking change:

- schema version bump
- migration plan (how old artifacts convert)
- compatibility window (how long both versions supported)

- updated registries (template/std/gate/version)
- updated docs + change impact notes
- updated golden fixtures + end-to-end run tests

C) Compatibility constraints

Your policy must state:

- what changes are backward-compatible (additive fields)
- what changes require a major bump (field removal/renames, semantics changes)
- how aliases are handled (IDs/terms preserved with mapping)

7.4 Security + IP policy

This is where you protect the repo, the kit, and your legal risk surface.

A) Plagiarism / reuse policy (internal + external agents)

Minimum requirements:

- **reuse allowlist** exists for any direct reuse of internal assets/examples
- **reuse log** exists if reuse happens
- external sources require license confirmation + attribution if needed
- knowledge library content can be used as *rules/patterns*, not copied into outputs unless allowlisted

B) Secrets handling policy

Rules must define:

- secrets never included in kits
- secrets never written to logs
- secrets redaction rules (tokens, keys, credentials)
- approved secret storage method (env vars, secret manager, CI secrets)

C) Data handling policy (intake + archives)

Define:

- PII classification tags
- retention windows per class (keep/prune/redact)
- archival rules (what must be scrubbed before long-term storage)
- access rules (who can view intake artifacts, especially external agents)

Tab 37

Tab 38

Tab 39

Tab 40

Runtime module coverage map

Runtime module coverage map

Proves the docs are backed by real code and that the system is actually executable.

8.1 Doc → Runtime mapping

- **DOC_RUNTIME_MAP**: for each doc ID (SYS/INT/CAN/STD/TMP/ORD/PLAN/VER/KIT/STATE/GOV/EXEC), list:
    - owning runtime module(s)
    - CLI command(s) / API entrypoint(s)
    - produced artifacts
    - required tests/proofs

8.2 Capability status tracking

- **CAPABILITY_STATUS_LIST**: each capability marked:
    - implemented / partial / stubbed / deprecated
- **STUB_DETECTION_RULES**: definition of what counts as “stubbed” (NotImplemented, TODO-throw, missing handler)

8.3 Coverage requirements (what “covered” means)

- **Declared**: mapped to a runtime owner
- **Runnable**: can be executed via CLI/API and produces artifacts
- **Proven**: has tests or proof evidence and passes gates

8.4 Coverage gates

- gates that fail if:
    - required runtime pieces are missing

- “stubbed” code exists in active paths
- required proofs/tests for a capability are missing

Tab 88

Tab 89

Tab 90

Tab 91

Run profiles and build recipes

Run profiles and build recipes

A deterministic “recipe layer” that selects the right standards/templates/gates/stages for a given build intent.

9.1 Build/run profile contract

- **RUN_PROFILE**: defines:
    - active stages
    - active gate set
    - default standards packs
    - template selection profile (or direct template set)
    - required outputs (artifacts + proofs)

9.2 Profile resolution rules

- **PROFILE_RESOLUTION_RULES**: deterministic selection when multiple profiles apply:
    - priority order
    - tie-breakers
    - overrides allowed (and how recorded)

9.3 Profile variants (common)

- prototype / standard / hardened / compliance
- internal agent vs external agent execution profiles
- mode-driven overrides (MM-xx modifies profile)

Tab 92

Tab 93

Tab 94

Control plane lifecycle specification

Control plane lifecycle specification

Makes the system governable as an actual runtime (not just scripts).

10.1 Run state machine

- **RUN_STATE_MODEL**: states and allowed transitions
    - queued → running → gated → failed → released → archived (example)
- **STAGE_TRANSITION_RULES**: what can advance, rerun, rollback, or resume

10.2 Persistence rules

- **STATE_PERSIST_POLICY**: what must be written at each transition:
    - run manifest
    - stage report
    - gate report
    - proof ledger updates
    - snapshots/pinsets

10.3 Operator actions and audit coupling

- **OP_ACTIONS_MODEL**: start/stop/resume/override/approve
- **AUDIT_BINDING_RULES**: every operator action writes an audit record with required fields

10.4 Failure semantics

- **HARD_STOP_RULES**: what failures block progression
- **RETRY/RERUN_POLICY**: safe reruns, idempotency expectations

Tab 96

Tab 97

Tab 98

Tab 99

Execution security boundary model

Execution security boundary model

The mechanical constraints that make “internal vs external agent” safe and enforceable.

11.1 Capability sandboxing

- **EXECUTION_SANDBOX_RULES**: per executor type:
    - allowed filesystem roots
    - allowed network access
    - allowed commands/tooling
    - secrets access rules

11.2 Data boundary rules

- **DATA_CLASSIFICATION_RULES**: PII/secret/internal/public classifications
- **REDACTION_POLICY**: what must be removed from:
    - logs
    - kits
    - archives
    - proof objects

11.3 External agent restrictions

- **EXTERNAL_AGENT_CONSTRAINTS**:
    - no access to knowledge library unless explicitly allowed
    - reuse allowlist required
    - minimal artifact exposure in kits (principle of least privilege)

11.4 Security gates

- gates that fail if:
    - disallowed paths are accessed
    - secrets/PII appear in logs or kits
    - executor capability claims don’t match actual behavior

Tab 100

Release and distribution discipline

Release and distribution discipline

Enterprise-grade supply-chain guarantees for shipped kits and runtime releases. Add only if you want tamper resistance, provenance, and formal release governance.

12.1 Release artifact set

Defines what constitutes a “release” and what must be produced.

- **RELEASE_MANIFEST**: release_id, included bundle_ids, pinset, schema versions, gate set used, compatibility notes
- **RELEASE_NOTES**: changes, breaking changes, migrations, deprecations
- **RELEASE_ATTESTATIONS** (optional but common): provenance metadata for how the bundle was produced

12.2 Signing and integrity

Ensures distributed bundles/manifests can’t be tampered with silently.

- **SIGNATURE_POLICY**: what must be signed (bundle, manifest, pinset, gate report)
- **KEY_MANAGEMENT_RULES**: key storage, rotation, revocation, who can sign
- **INTEGRITY_CHECK_RULES**: verify signatures/hashes on import and repro

12.3 Provenance and SBOM (optional but strong)

Makes builds auditable at the dependency level.

- **PROVENANCE_RECORD**: toolchain versions, environment fingerprint, build commands, source commit refs
- **SBOM**: dependency inventory (runtime + build-time)
- **DEPENDENCY_RISK_POLICY**: denylist/allowlist, license compatibility, CVE thresholds

12.4 Distribution channels and access control

Controls who gets which bundles and what they contain.

- **DISTRIBUTION_RULES**: internal vs external bundle variants, redaction rules per channel
- **ACCESS_CONTROL_POLICY**: who can download/export bundles, logging of access
- **RETENTION_POLICY (release-level)**: retention windows for released artifacts

12.5 Release gates

Release-only gates that must pass before anything is labeled “released.”

- **GATE-REL-01**: all required stage artifacts present + pinned
- **GATE-REL-02**: reproducibility check passes (pack + repro match)
- **GATE-REL-03**: signatures valid (if signing enabled)
- **GATE-REL-04**: SBOM/provenance present (if required)
- **GATE-REL-05**: IP/security checks pass (reuse log + secrets scan)

Tab 101